

SmartFlow Verify

TOC Case Study Defense & Project Guide (Easy Language)

Authors: Tanmay Kapoor (24BIT0079) | Chinmay Agrawal (24BIT0453)

Theory of Computation (TOC) • Finite State Automata • Formal Methods

1. Simple Project Summary (What is SmartFlow Verify?)

SmartFlow Verify is a software platform that converts business processes (like banking loan approvals, healthcare claims, or employee leave management) into mathematical **Finite State Machines (FSMs)**.

Instead of guessing whether a business workflow has hidden bugs, our system automatically runs **graph algorithms (BFS/DFS)** to mathematically prove:

- **No Deadlocks:** Processes never get permanently stuck in a trap state.
- **100% Reachability:** There are no unused (dead) states.
- **Determinism:** For any user action, the system always knows exactly which next state to transition to.
- **1-Click Auto-Repair:** Automatically creates missing transitions to fix broken workflows.

Key Takeaway for Faculty: "We use Theoretical Computer Science (Finite Automata & Graph Theory) to guarantee 100% correctness of business logic before it is deployed into production."

2. How SmartFlow Verify Directly Connects to TOC (Theory of Computation)

A. The Formal 5-Tuple Definition of a Deterministic Finite Automaton (DFA)

In Theory of Computation, every Finite State Machine is defined by the mathematical 5-tuple:

$$M = \langle Q, \Sigma, \delta, q_0, F \rangle$$

Here is how our project implements each of these 5 parts in simple terms:

TOC Symbol	Mathematical Definition	SmartFlow Verify Implementation
Q (States)	Finite set of all states in the system	Every step in the workflow (e.g. Application_Submitted, Manual_Review).
Σ (Alphabet)	Set of all input symbols (events)	All user triggers and actions (e.g. submit_docs, manager_approve).
δ (Transition Function)	Mapping: $\delta: Q \times \Sigma \rightarrow Q$	The arrows connecting states with boolean guards (e.g. [creditScore >= 700]).
q₀ (Initial State)	The unique start state ($q_0 \in Q$)	The entry node where a new application starts (e.g. Application_Submitted).
F (Final / Accepting States)	Subset of terminal states ($F \subseteq Q$)	The final outcome nodes (e.g. Disbursed & Active or Application_Rejected).

B. Language Recognition & String Acceptance: L(M)

- **Accepted Language L(M):** A sequence of operations $w = \sigma_1 \sigma_2 \dots \sigma_k$ is accepted if starting from q_0 , following the transitions reaches an accepting final state $q_{final} \in F$.
- **Rejected String:** If an action leads to a deadlock state or an invalid branch, the string is rejected (the transaction failed).

C. Formal Algorithms Implemented in Code

TOC Problem	Algorithm Used	Time Complexity	Why It Matters
Unreachable State Detection	Forward Breadth-First Search (BFS) starting from q_0	$O(V + E)$	Finds orphaned states that no user can ever reach.
Deadlock / Trap State Detection	Sink state detection & Reverse BFS from final states F	$O(V + E)$	Finds states with $out_degree = 0$ that are not in F (stranded items).
Determinism Verification	Alphabet conflict checking: $ \delta(q, \sigma) \leq 1$	$O(Q \times \Sigma)$	Guarantees system is a DFA with zero conflicting paths.
Transition Function Table	Cartesian Product Matrix: $Q \times \Sigma$	$O(Q \times \Sigma)$	Displayed under State Matrix (δ) tab in UI.

3. Verification Against Your Case Study Report (100% Match)

Our implementation directly implements all 5 objectives stated in your report:

1. **Design business workflows using FSMs:** Interactive drag-and-drop canvas with automatic topological auto-layout.
2. **Automatically verify workflow:** Real-time formal verification engine checking deadlocks, unreachables, determinism, and SLAs.
3. **Suggest correction for detected errors:** Automated rule-based synthesis that fixes deadlocks with 1-click Auto-Repair.
4. **Provide step-by-step workflow simulation:** Interactive Token Simulator with speed controls (0.5x, 1x, 2x) and execution trace history.
5. **Cloud Persistence & User Data:** Real-time sync with Google Cloud Firestore and Firebase Authentication.

4. Tab-by-Tab Explanation for Faculty Presentation

1. FSM Graph Tab (Canvas): Visualizes the directed graph with color-coded nodes (Initial, Review, Gateway, Terminal). Displays transition arrows with trigger events and conditional boolean guards.

2. Verification Audit Tab (Formal Dashboard): Shows the mathematical **Reliability Score (0-100)**, lists detected defects, displays the exact **Counter-Example Trace** leading to failures, and provides a 1-click **Auto-Repair** button.

3. State Matrix (δ) Tab: Displays the formal transition function table $\delta(q, \sigma)$ mapping all state-event pairs, exactly as taught in Theory of Computation textbooks.

4. Token Simulator Tab (Execution Engine): Allows step-by-step token execution. Includes a **Live Guard & Context Payload Editor** where modifying variables (e.g. `creditScore`) dynamically evaluates guards into **GUARD PASS (Green)** or **BLOCKED (Red)**. Also includes a **1,000-Run Monte Carlo Stress Tester** for statistical SLA calculations.

5. Cloud Library & Export: Saves workflow models to Google Cloud Firestore with Firebase Auth. Exports verified FSMs to **Temporal.io (TypeScript)**, **AWS Step Functions (ASL JSON)**, and **XState v5**.

5. Likely Faculty Viva Questions & Exact Answers

Q1: How is your project related to Theory of Computation?

Answer: "Our project models business logic as a formal Deterministic Finite Automaton (DFA) defined by the 5-tuple $M = \langle Q, \Sigma, \delta, q_0, F \rangle$. We apply graph traversal algorithms (BFS/DFS) on the transition graph to check language acceptance $L(M)$, detect unreachable states, and eliminate deadlock sink states."

Q2: How do you mathematically detect a deadlock?

Answer: "A deadlock occurs when a state q is reachable from q_0 , but has an out-degree of 0 (no outgoing transitions) and is not a member of the final accepting states set F . In automata theory, this is known as a dead state or trap state."

Q3: What is the time complexity of your verification algorithms?

Answer: "Reachability and deadlock analysis run in $O(|V| + |E|)$ time, where $|V|$ is the number of states and $|E|$ is the number of transitions. This is significantly faster and more scalable than Petri-Net reachability graphs which suffer from state space explosion (exponential complexity)."

Q4: How does the Auto-Repair feature work?

Answer: "When a deadlock state is detected, the synthesis engine automatically generates a fallback transition from the stranded state to an absorbing terminal rejection state with an investigation timeout guard, restoring formal liveness to the FSM."